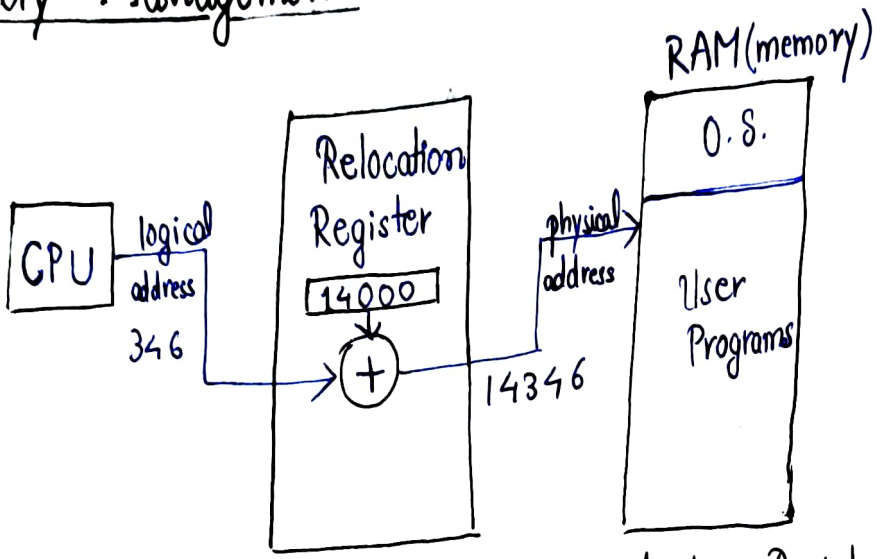
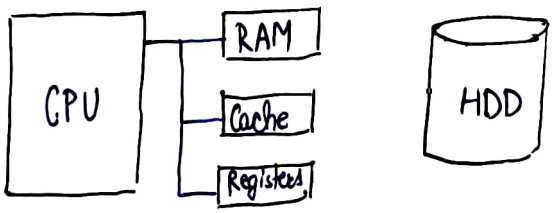
Memory Management



Dynamic Relocation using Relocation Register



- OS maintains a table of the processes and their base addresses.

Pid	B.A.
P1	20000
P2	12264

- When a program is run, its base address is loaded in the relocation register.
- CPU generates logical addresses which the MMU converts to physical address.
  (memory management unit)
- If program size is bigger than available RAM, we use techniques like overlaying.

Overlay

Memory Space: 150 KB

Assembler:

Pass 1	70 KB
Pass 2	80 KB
Common Routine	20 KB
Symbol Table	30 KB
	200 KB

Overlay-A	Overlay-B
Pass 1	Pass 2
CR	CR
ST	ST
120 KB	130 KB
+ 10	+ 10

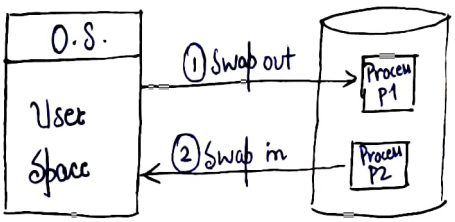
Overlay Driver — 10 KB → memory overhead

- → first loads Overlay-A (120 KB)
- → after that loads Overlay-B (130 KB)

* Advantages:
  - We can run programs whose size exceeds the RAM size
  - Each overlay is loaded faster (as it doesn't include the full program)

* Disadvantages:
  - There is extra memory overhead, due to Overlay Driver.
  - Execution time is increased as we need to load CR and ST twice

Swapping

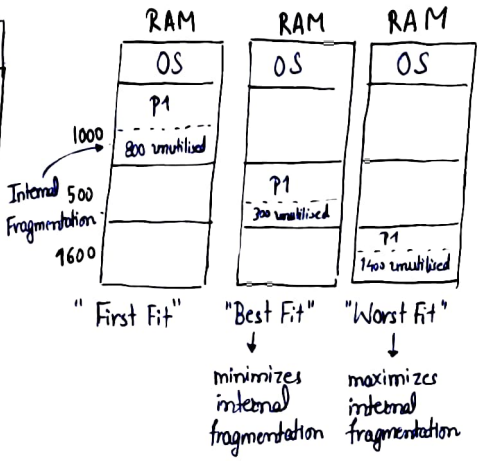


- Used in multi-programming system when RAM is NOT sufficient for all the running processes.
- A higher priority process can preempt a lower priority one, swapping it out of RAM to create vacant space for it to swap in.

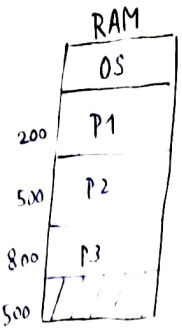
Memory Allocation Techniques

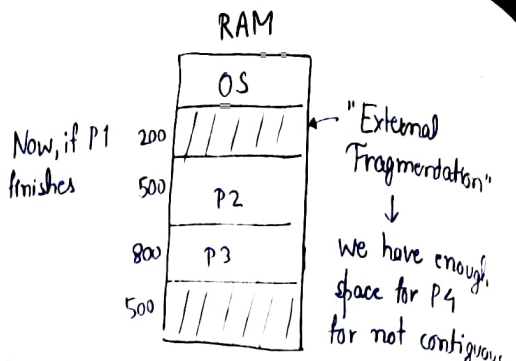
- Contiguous
  - Fixed Partition
    - → First Fit
    - → Best Fit
    - → Worst Fit
  - Dynamic/ Variable Partition
- Non-contiguous
  - → Paging
  - → Segmentation

Pid	Size
P1	200
P2	500
P3	800



Process	Size
P1	200
P2	500
P3	800
P4	700

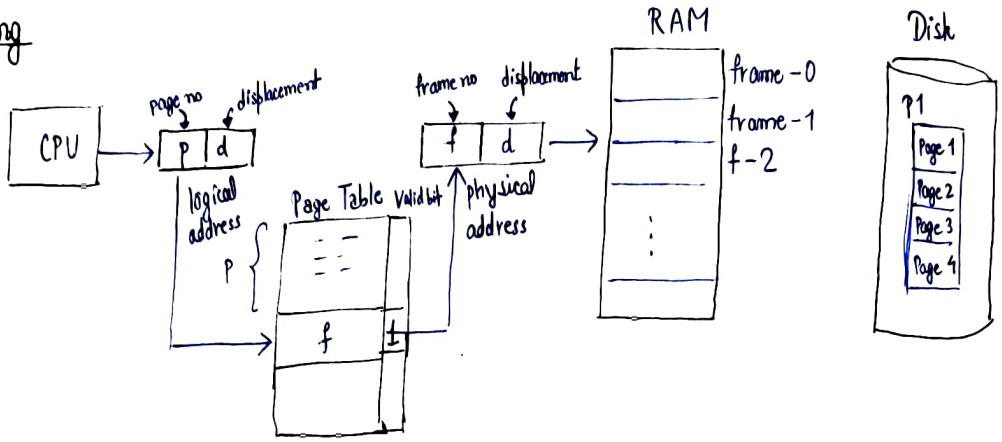




Dynamic Partitioning

* Compaction: shifting the memory blocks to make free memory contiguous to avoid external fragmentation.

Paging

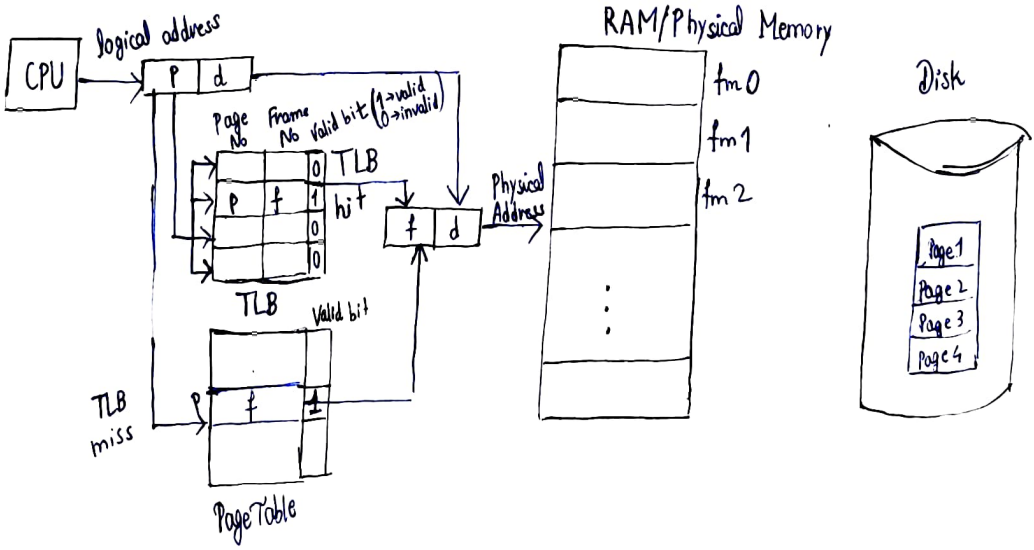


Paging Hardware

- Program is divided into fixed size "pages" and RAM is divided into the same fixed size "frames".
- Each page is loaded into some frame (not necessarily contiguous).
- Each process has a "page table" tracking which page is loaded in which frame.

* Logical Address: Page no. + displacement
* Physical Address: Frame no. (from page table) + displacement

Translation Look-aside Buffer (TLB)



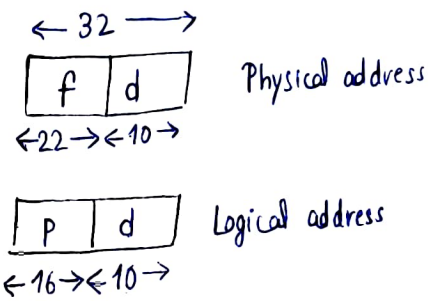
- Page Table is also stored in Primary memory, so we've to access the primary memory twice — once to access the Page Table and once to access the actual physical address.
- So, we use TLB as a cache memory to store a subset of the pages that have been recently used for faster access.
  - * TLB hit: when page table entry is found in TLB
  - * TLB miss: when page table entry is NOT found in TLB

Q.) Consider a simple paging system with the following parameters:

- 2^{32} bytes of physical memory
- 2^{10} bytes of page size
- 2^{16} pages of logical address space

1. How many bits are there in logical address?

Ans→ 26 bits



2. How many bytes are there in a frame?

Ans→ 2^{10} bytes = 1 kB (page size = frame size)

3. How many bits specify frame no?

Ans→ 22 bits

4. How many entries are there in a page table?

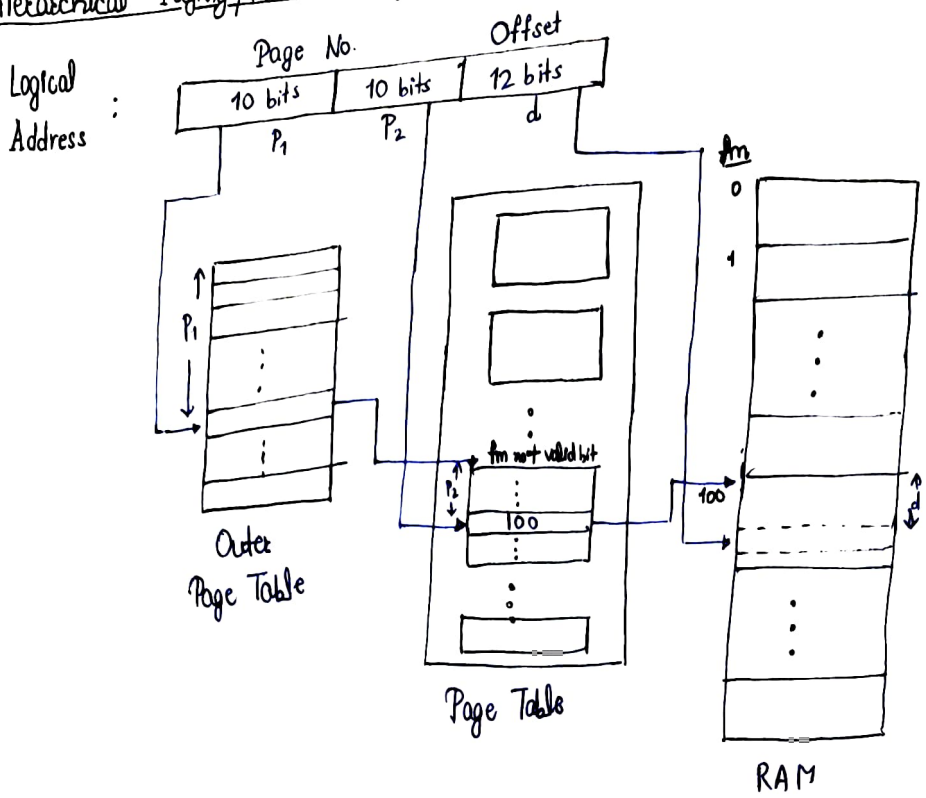
Ans→ 2^{16} entries.

- Effective Memory Access Time when we use TLB:

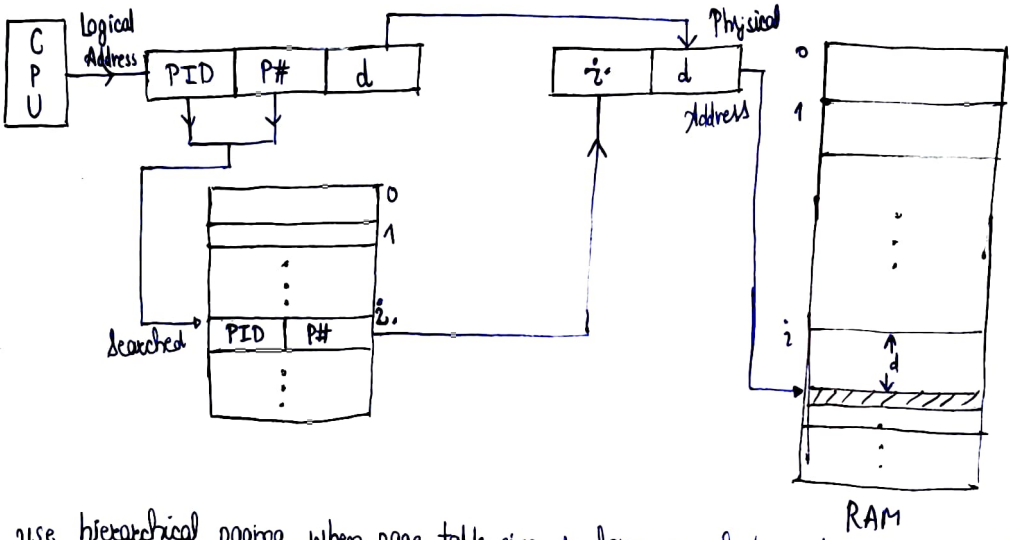
$$\boxed{EMAT = h \times (c+m) + (1-h) \times (c+2m)}$$

- h → hit ratio
- c → cache memory access time
- m → main memory access time

Hierarchical Paging / Multi-Level Paging



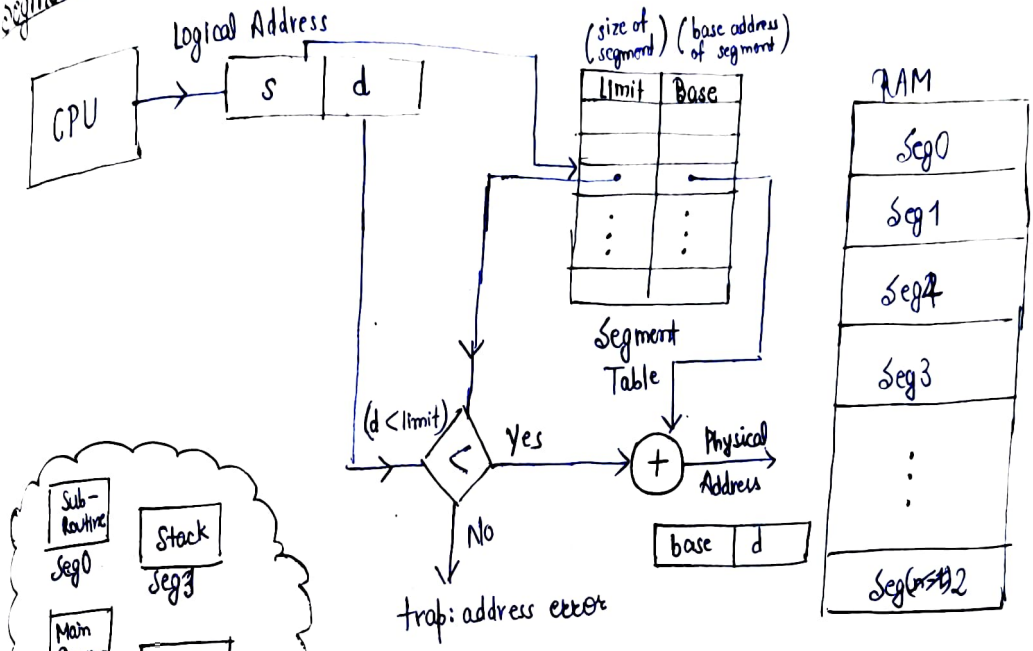
Inverted Page Table

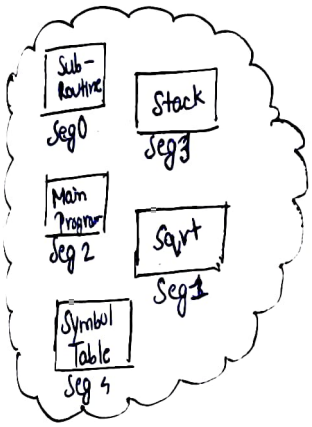


- We use hierarchical paging, when page table size is large enough to not be accomodated in a single frame.
- Since we require multiple frames, they'll likely not be contiguous so we use an outer page table to keep track of them.
- Larger page table is required when logical address space is large (allowing larger processes).
- In the above case, we need to access main memory thrice to read a byte.
- Here, if RAM is divided into 2^{23} frames:
  - 2^{20} pages entries (20 bits in page no)
  - (23+1) bits stored in each entry (2^{23} frames + 1 valid bit)
  - $2^{20} \times 3$ bytes is the size of page table (each page entry stores 24 bits info)
  - $\frac{2^{20} \times 3}{2^{12}}$ ~~bytes~~ frame size is the no. of frames required to store each page table (each process has its own page table)
- Max^m size of Process here = 2^{32} bytes = 4 GiB (32 bits → logical address size)

- In inverted page table, page table size is limited by no. of frames in main memory.
- Only 1 page table for all processes in this process.
- One entry for each frame in page table.
- All entries of the page table need to be searched to find a match.
- Hashing technique can be used to reduce search time

Segmentation



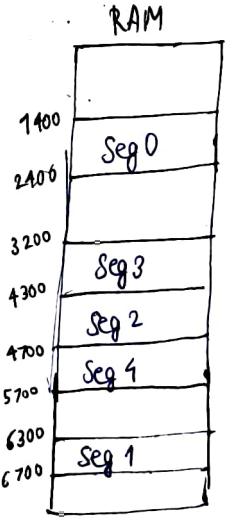


Program e.g.-

Segmentation Hardware

Segment Table

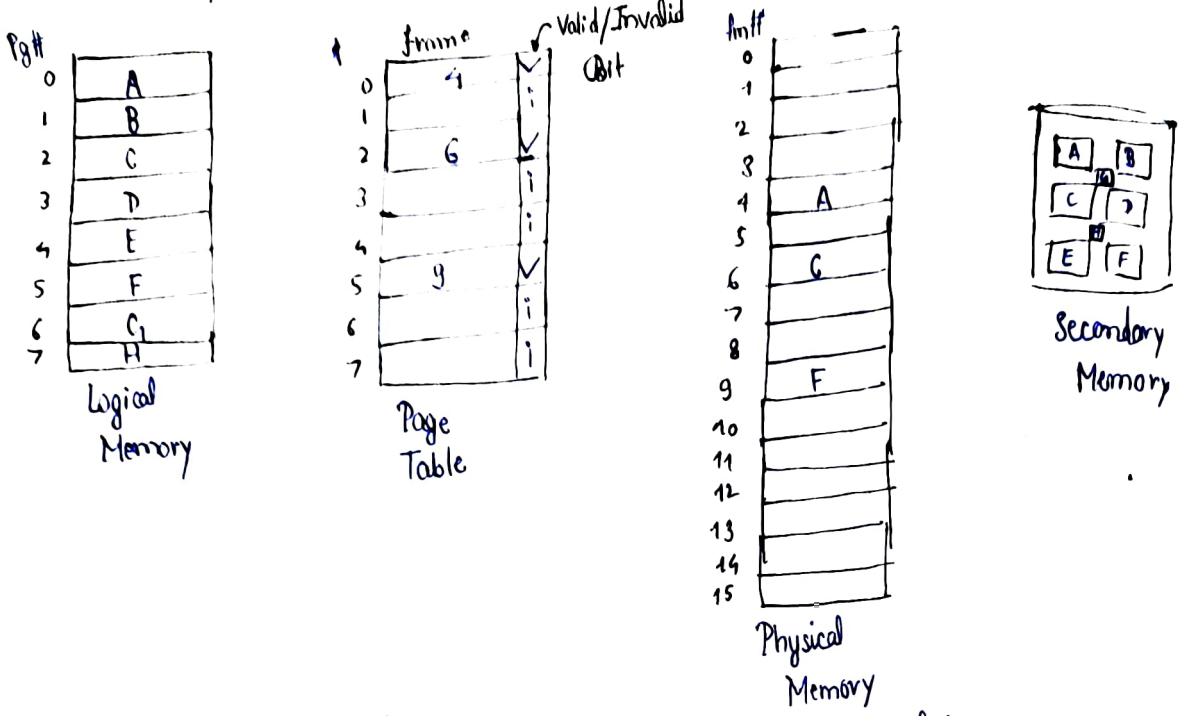
	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700



- Paging may suffer from internal fragmentation while segmentation may suffer from external fragmentation.
- Size of the segments are NOT fixed.

* In Paging and Segmentation, whole program need to be loaded in RAM before execution

Virtual Memory



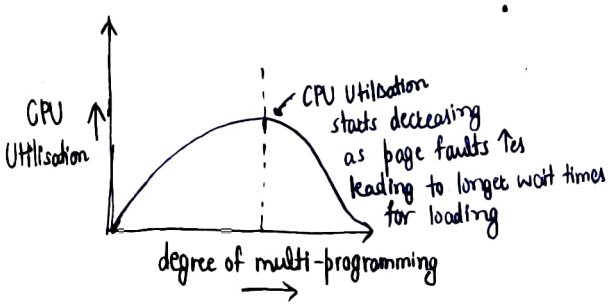
- Virtual Memory allows us to write programs larger than the Physical Memory.
- Only loads certain pages at a time in the RAM.
- If a page is requested that is NOT present in RAM, it is known as "Page Fault".
- When there is need of a page, we load it from secondary to primary memory, it is called "On-Demand Paging".
- If Primary Memory is full, we call "Page Replacement Algorithms".
  - → FIFO (First In First Out)
  - → LRU (Least Recently Used)
  - → Optimum (page referred late in the future is replace)

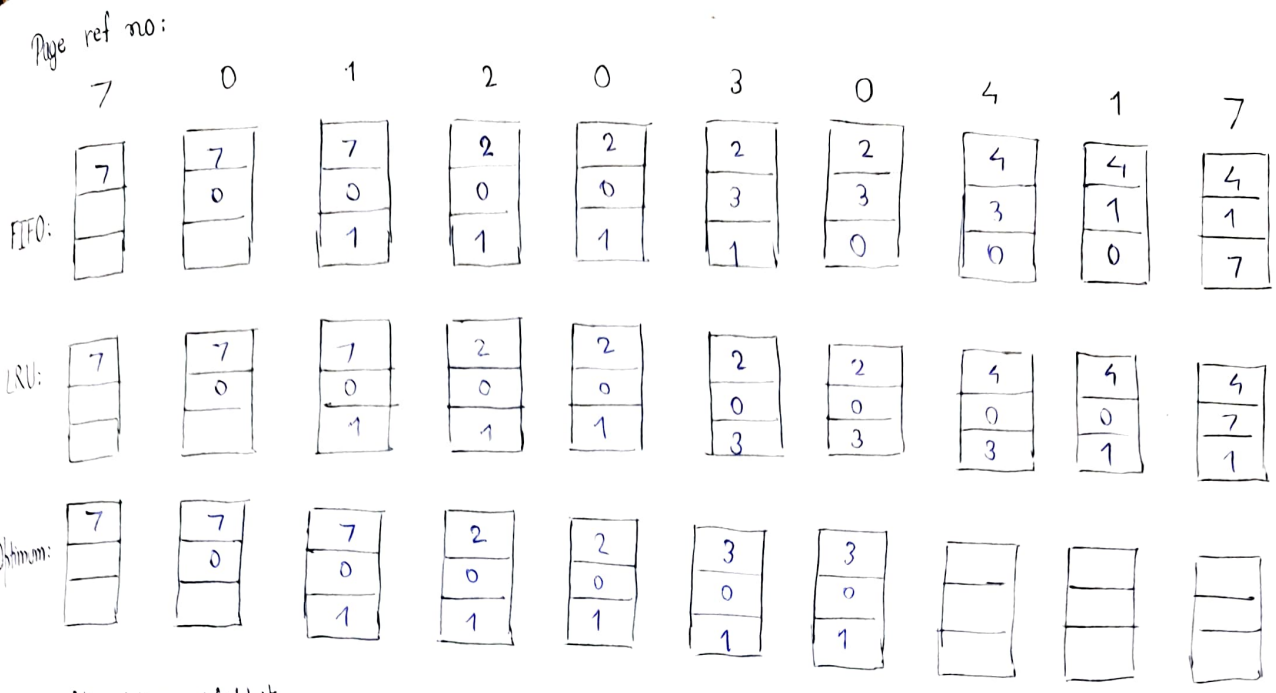
Effective Memory Access Time = (1-p) * memory access time + p * page fault time

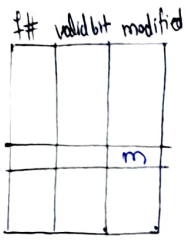
p: probability of page fault

page fault time: time needed to handle page fault

* Thrashing:





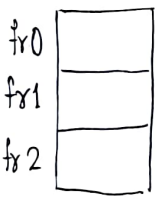


Page Table

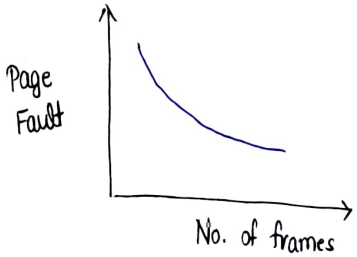
→ We add an extra modified bit in page table to reduce effective memory access time (so that we do not need to write that page back to secondary memory is it is NOT modified)

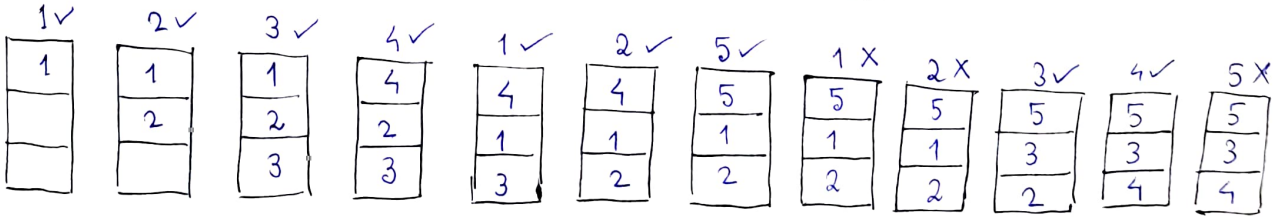
Balad's Anomaly (FIFO Page Replacement)

Page reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

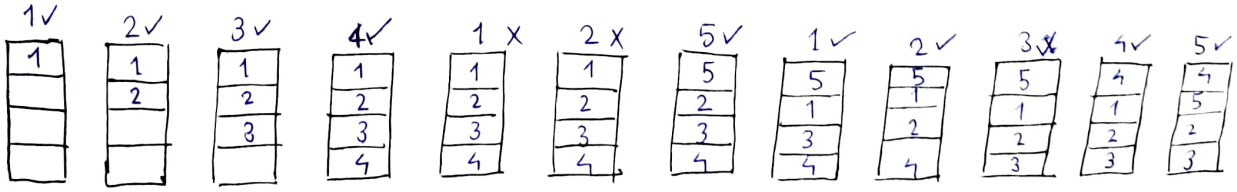


RAM





- 9 Page faults



- 10 Page faults

* This anomaly is observed sometimes in FIFO page replacement known as Balad's anomaly.

* Working Set of a Process: set of pages in RAM that process is currently using
* Working Set Model: used to decide how many pages each process should have in memory at any given time (to avoid thrashing)
* Working Set Window: fixed no of recent memory references or a fixed time span

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Δ = 4 Δ = 4 Δ = 4

t_1: WS = {1, 2, 3, 4}

t_2: WS = {1, 2, 5}

t_3: WS = {2, 3, 4, 5}

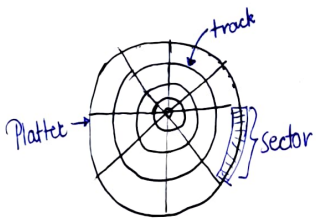
Page Replacement Techniques

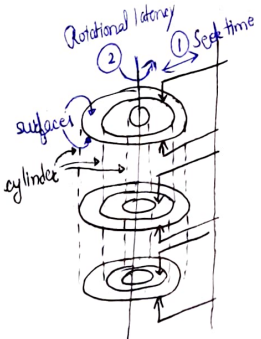
- Local → replacement only allowed for pages of that process
- Global → replacement allowed of any page of any process.

P_1^4
P_1^5
P_2^3
P_2^4

* Thrashing can be reduced by using local page replacement technique

Disk Scheduling



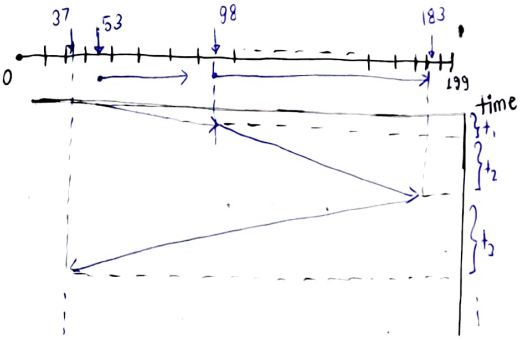


- Seek Time: time taken by disk arm to move head to the required track
- Rotational Latency: time taken by the desired sector to rotate under the head

Reference track no.: 98, 183, 37, 122, 14, 124, 65, 67

Initial head position: 53

① FIFO/FCFS:



Seek Time = (98-53) + (183-98) + ... + (67-65) = 640

(ii) SSTF (Shortest Seek Time First):

$53 \to 65 \to 67 \to 37 \to 14 \to 98 \to 122 \to 124 \to 183$

Seek Time $= (65-53) + \ldots + (183-124) = 236$

(v) LOOK

(iii) SCAN

(vi) C-LOOK

(iv) C-SCAN

Q.) Consider a disk with following specs: 20 surface, 1000 tracks/surface, 16 sectors/track, data density 1 kB/sector, rot^n speed 3000 rpm. The OS initiates transfer b/w disk and memory sector-wise. Once head has been placed on right track, the disk reads sector in a single scan. It reads bits from sector while head is passing over the sector

(a) What is total capacity of disk?

$20 \times 1000 \times 16 \times 1 \text{ kB} = 320000 \text{ kB}$

(b) What is data transfer rate?

$3200 \times 16 \times 1 \text{ kB/min}$

(c) How many bits will be used to identify a sector in the disk?

16 sectors/track $= 2^4$ sectors/track $= 4$ bits

20 surfaces $= 5$ bits

1000 tracks/surface $= 10$ bits

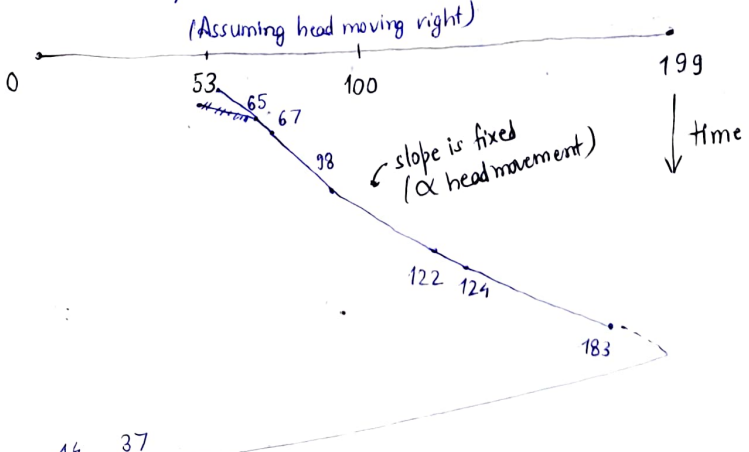
$\therefore$ No. of bits to uniquely identify each sectors $= 19$ bits

Reference tracks: 98, 183, 37, 122, 14, 124, 65, 67

Initial head position: 53

(iii) SCAN

· In this algorithm, the disk scans from innermost to outermost track and vice-versa and serves anything it encounters on its way



Seek Time $= (65-53) + (67-65) + (98-67) + \ldots + (183-124) + (187-37) + (37-14) =$

(IV) LOOK

- In this, instead of going always to outermost and innermost tracks it goes this only till the tracks where the final request is in that dirⁿ, then reverses dirⁿ

(V) C-SCAN

- In this, after going to outermost track, it returns back to the innermost track (not servicing any request in between)

(VI) C-LOOK

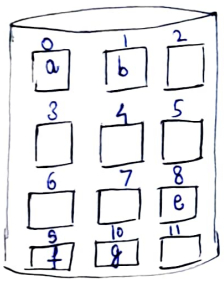
- In this, after going to outermost track need to be served, it returns to the innermost track need to be served (NOT servicing any request in between).

* In C-SCAN and C-LOOK, going from outermost to innermost track introduces some extra latency that must be added to the seek time.

File Allocation Methods

- ① Contiguous allocation
- ② Linked allocation
- ③ Indexed allocation

①



Directory:

File	Start	Length
count	0	2
tr	8	3
⋮	⋮	⋮

count File:

a
b

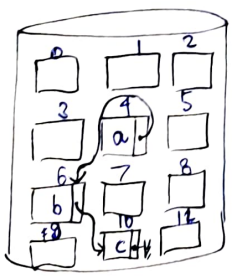
tr file:

e
f
g

* Advantages:
  - Random access is possible
  - Access time is less (less head movement)

* Disadvantages:
  - External fragmentation may arise
  - Insertion/deletion is difficult
  - Size of file can't grow easily

②



Directory:

File	Start	End
Jeep	4	10
⋮	⋮	⋮

Jeep File:

0	a
1	b
2	c

* Advantages:
  - External fragmentation doesn't occur
  - Size of file can be grown easily

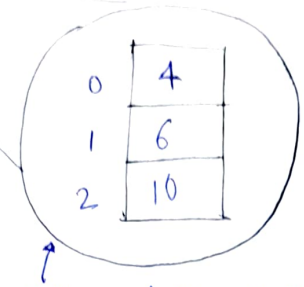
* Disadvantages:
  - Random access isn't possible
  - Access time is high
  - Memory overhead is there (storing pointers to the next block).

③



Directory:

File	Index-Block
Jeep	8



Mapping of file-index block to actual block in harddisk.

* Advantages:
  - Random access is possible
  - External Fragmentation won't occur

* Disadvantages:
  - Extra index-block need to be allocated for every file (memory overhead)
  - Access time will be high
  - For larger files, multiple index blocks will be required, so multi-level indexing is used (even higher access time)

Directory Implementation

→ ① Linear List
→ ② Hash Table

①

File	Attributes
f_1	...
f_2	...
f_3	...
f_4	...
⋮	...
.	...

- Can be stored as an array or a linked list.
- if we create a file, it first sequentially searches if that file already exists, if NOT we create it
- if we delete a file, it first sequentially searches if that file exists, if it does we remove it

②

Hash Table

0	f_3	... (linear probing)
1		
2		
3	f_7	... → f_3 ... (chaining)

*Hash Function(f_i): $i \bmod 4$

- To create file f_7, we hash it: $7 \bmod 4 = 3$, then search only the 3rd index to see if the file exists, if NOT it creates it
- f_3: $3 \bmod 4 = 3$, so it checks index 3 and finds f_7 there → "Hash Collision"

* To avoid collisions, we use techniques like:
  i) Linear Probing — we search the next index which is free (wrap around if at end)
  ii) Chaining — we create a linked list at the index

- If hash table gets full, we double the size, change the hash function accordingly and re-hash the existing values.